

GPU-as-a-Service

Architecture distribuee avec Proxmox et Cricket

Plan d'action complet | Implementation en C

Proxmox 9.x

GTX 1650 Ti

C pur

1. Contexte et objectif

Ce projet transforme un GPU physique en un service reseau partage, accessible simultanement par plusieurs machines virtuelles sans passthrough ni vGPU. L'approche retenue est le GPU Remoting via Cricket (CUDA over network), combinee a un scheduler personnalise inspire de la logique Credit de Xen.

Host	Proxmox VE 9.x sur machine physique
GPU	NVIDIA GTX 1650 Ti Mobile (TU117M - architecture Turing)
CUDA Remoting	Cricket (open source, RWTH-ACS) — remplace rCUDA
Scheduler	Logique Credit de Xen, adaptee au GPU, implementee en C
Distribution	Package .deb installe sur le host Proxmox
Langage	C pur (C99/C11) pour tous les composants du projet
VMs cibles	Ubuntu 22.04 ou 24.04, locales ou distantes

2. Architecture globale

Le stack est compose de 7 couches distinctes, chacune avec un role precis et des technologies bien definies. Aucun composant ne depend d'un autre langage que le C.

Couche	Technologie	Langage
VMs clientes	Proxmox KVM (Ubuntu 22/24)	App CUDA existante, pas de modification
CUDA Remoting	Cricket (cricket-client.so)	C — LD_PRELOAD, intercepte libcuda.so
GPU Scheduler	Daemon C personnalis�	C — logique Credit Xen, POSIX sockets
MPS NVIDIA	CUDA Multi-Process Service	Daemon NVIDIA — % compute par processus
GPU physique	GTX 1650 Ti (driver 550.x)	Driver propri�taire NVIDIA
Package	gpu-remoting.deb	Structure Debian, systemd service
Monitoring	Prometheus exporter	C — exposition metriques HTTP

Flux de traitement

Le chemin d'un appel CUDA depuis une VM jusqu'au GPU physique :

```

VM (app CUDA)
-> cricket-client.so [interception LD_PRELOAD]
-> TCP/IP [serialisation appel CUDA]
-> cricket-rpc-server [reception host]
-> gpu-scheduler [Credit engine : UNDER/OVER/IDLE]
-> CUDA MPS server [% compute GPU par VM]
-> GPU physique [execution kernel CUDA]
-> retour resultat [meme chemin inverse]
```

3. Composants internes

3.1 Cricket (CUDA Remoting)

Cricket est la fondation du projet. Il remplace rCUDA qui est abandonn  et ne supporte que CUDA 9.0 (incompatible Turing/Ampere).

- Repository : github.com/RWTH-ACS/cricket
- Licence : open source, libre d'utilisation
- Principe : remplace libcuda.so via LD_PRELOAD
- Transport : TCP/IP (par default) ou RDMA InfiniBand
- Compatibilit  : CUDA 12.x, pas de recompilation des apps
- Checkpoint/Restart : supporte nativement

Commandes de base apres compilation :

```
# Serveur (host Proxmox)
/opt/cricket/bin/cricket-rpc-server

# Client (VM)
REMOTE_GPU_ADDRESS=<IP_HOST> \
LD_PRELOAD=/opt/cricket/bin/cricket-client.so \
<binaire-cuda>
```

3.2 GPU Scheduler (logique Credit Xen)

C'est le composant central du projet, implemente entierement en C. Il s'inspire du scheduler Credit de Xen qui gere le partage CPU entre VMs avec deux parametres par domaine : weight et cap.

Parametres par VM

- weight : part relative du GPU (default 256, comme Xen)
 - VM avec weight 512 obtient deux fois plus de temps GPU qu'une VM avec weight 256
 - Les credits sont distribues proportionnellement au weight a chaque recharge
- cap : pourcentage maximum du GPU utilisable (0 = illimite)
 - cap = 60 : la VM ne peut jamais depasser 60% du GPU meme si idle
 - cap = 0 : mode work-conserving, utilise les cycles libres

Etats des VMs (comme Xen)

- UNDER : credits positifs -> VM autorisee a soumettre des jobs
- OVER : credits epuises -> VM mise en file d'attente
- IDLE : pas de job en cours -> credits non debites

Cycle de fonctionnement

```
Toutes les 30ms (comme Xen) :
1. Recharger credits de toutes les VMs (proportionnel au weight)
2. Trier file : UNDER avant OVER, puis par credits decroissants
3. Executer le job de la VM en tete
4. Debiter credits utilises
5. Si credits < 0 -> passer en etat OVER
6. Verifier cap -> bloquer si quota atteint
```

Technologies C utilisees

- pthreads : thread de recharge periodique des credits
- POSIX sockets (epoll) : reception des jobs depuis Cricket
- Socket UNIX : communication avec gpu-cli
- timerfd : declenchement precis du cycle de recharge
- libyaml ou parsing manuel : lecture config.yaml

3.3 CUDA MPS (Multi-Process Service)

CUDA MPS est un daemon NVIDIA qui permet a plusieurs processus de partager le GPU simultanement avec un controle fin du pourcentage de compute. Le scheduler s'appuie sur MPS pour appliquer les caps.

- Gere par : nvidia-cuda-mps-control et nvidia-cuda-mps-server
- Permet de fixer CUDA_MPS_ACTIVE_THREAD_PERCENTAGE par client
- Transparent pour les applications CUDA

3.4 gpu-cli

Outil de gestion en C communiquant avec le scheduler via socket UNIX.

```
gpu-cli status    # etat global du scheduler
gpu-cli top      # VMs actives, credits, etat
gpu-cli jobs     # file d'attente actuelle
gpu-cli set <vm> weight=512 cap=60    # modifier params
```

3.5 Package .deb

Tout le projet est distribue sous forme de package Debian installe sur le host Proxmox en une commande.

```
gpu-remoting/
DEBIAN/control
usr/bin/gpu-scheduler
usr/bin/gpu-cli
usr/lib/cricket/cricket-rpc-server
usr/lib/cricket/cricket-client.so
etc/gpu-remoting/config.yaml
lib/systemd/system/gpu-remoting.service
```

4. Configuration

Fichier de configuration central : /etc/gpu-remoting/config.yaml

```
gpu:
  devices: [0]          # index GPU

scheduler:
  policy: credit        # logique Xen Credit
  time_slot_ms: 10      # duree d'un slot
  recharge_interval_ms: 30 # recharge credits (comme Xen)

network:
  port: 5000
  bind: 0.0.0.0

vms:
  vm-101:
    weight: 512          # priorite double
    cap: 60              # max 60% GPU
  vm-102:
    weight: 256          # priorite standard
```

```
cap: 30 # max 30% GPU
vm-103:
weight: 256
cap: 0 # illimite (work-conserving)
```

5. Plan d'action par phases

Le projet est decoupage en 4 phases sequentielles. Chaque phase doit etre validee avant de passer a la suivante.

Phase	Nom	Objectif	Livrable
Phase 1	Installation host	Driver NVIDIA + Cricket operationnel sur Proxmox	nvidia-smi + test Cricket local
Phase 2	Test VMs	VMs Ubuntu executant du CUDA via Cricket	Benchmark CUDA depuis VM locale et distante
Phase 3	Scheduler C	Daemon Credit operationnel avec quotas par VM	gpu-cli status + metriques credit
Phase 4	Package .deb	Distribution complete et deployable	dpkg -i gpu-remoting.deb sur host vierge

Phase 1 — Installation du host Proxmox

Etape 1.1 — Blacklister le driver nouveau

```
echo 'blacklist nouveau' > /etc/modprobe.d/blacklist-nouveau.conf
echo 'options nouveau modeset=0' >> /etc/modprobe.d/blacklist-nouveau.conf
update-initramfs -u
```

Etape 1.2 — Installer le driver NVIDIA proprietaire

La GTX 1650 Ti (TU117M) necessite le driver serie 550.x. Le kernel Proxmox 6.17.x etant tres recent, on installe via le .run NVIDIA avec DKMS pour la recompilation automatique.

```
apt install -y build-essential gcc make perl dkms
wget https://us.download.nvidia.com/XFree86/Linux-x86_64/550.144.03/NVIDIA-Linux-x86_64-550.144.03.run
chmod +x NVIDIA-Linux-x86_64-550.144.03.run
./NVIDIA-Linux-x86_64-550.144.03.run --dkms
reboot
# Verification apres reboot :
nvidia-smi
```

Etape 1.3 — Installer Cricket sur le host

```
apt install -y git gcc make libtirpc-dev
cd /opt && git clone https://github.com/RWTH-ACS/cricket.git
```

```
cd cricket && git submodule update --init --recursive
LOG=INFO make
# Verification :
ls /opt/cricket/bin/ # cricket-rpc-server + cricket-client.so
```

Etape 1.4 — Test Cricket en local

```
# Terminal 1 : demarrer le serveur
/opt/cricket/bin/cricket-rpc-server &

# Terminal 2 : tester
REMOTE_GPU_ADDRESS=127.0.0.1 \
LD_PRELOAD=/opt/cricket/bin/cricket-client.so \
/opt/cricket/tests/test_kernel
```

Phase 2 — Configuration des VMs

Etape 2.1 — Créer une VM dans Proxmox

- OS : Ubuntu 22.04 ou 24.04
- RAM : 4 Go minimum
- Aucun GPU attache (principe fondamental)
- Réseau : bridge vmbr0 (reseau interne Proxmox)

Etape 2.2 — Préparer la VM

```
# Dans la VM : installer les libs CUDA (sans driver GPU)
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/
x86_64/cuda-keyring_1.1-1_all.deb
dpkg -i cuda-keyring_1.1-1_all.deb
apt update && apt install -y cuda-libraries-12-x libtirpc-dev
```

Etape 2.3 — Copier le client Cricket

```
# Depuis le host vers la VM
scp /opt/cricket/bin/cricket-client.so user@<IP_VM>:/opt/
```

Etape 2.4 — Tester depuis la VM

```
# Dans la VM
REMOTE_GPU_ADDRESS=<IP_HOST_PROXMOX> \
LD_PRELOAD=/opt/cricket-client.so \
./mon-binaire-cuda
```

Phase 3 — Développement du scheduler en C

Etape 3.1 — Structure des fichiers sources

```
gpu-scheduler/
src/
  main.c          # entree, boucle principale
  scheduler.c     # logique Credit (weight, cap, credits)
  scheduler.h
  vm_registry.c   # gestion des VMS enregistrees
```

```

vm_registry.h
job_queue.c      # file d'attente POSIX
job_queue.h
config.c          # parsing config.yaml
config.h
socket_server.c  # reception jobs depuis Cricket
socket_server.h
mps_bridge.c      # interface vers CUDA MPS
mps_bridge.h
cli_server.c      # socket UNIX pour gpu-cli
cli_server.h
metrics.c         # export Prometheus
metrics.h
Makefile

```

Etape 3.2 — Structures de donnees cles

```

/* Etat d'une VM dans le scheduler */
typedef enum { UNDER, OVER, IDLE } vm_state_t;

typedef struct {
    char    vm_id[64];
    uint32_t weight;      /* part relative, default 256 */
    uint32_t cap;         /* % max GPU, 0 = illimite */
    int64_t credits;      /* credits courants */
    vm_state_t state;     /* UNDER / OVER / IDLE */
    uint64_t gpu_usage_ms; /* usage cumule slot courant */
    pthread_mutex_t lock;
} vm_entry_t;

typedef struct {
    vm_entry_t *vms[MAX_VMS];
    int        vm_count;
    uint64_t    total_weight;
    pthread_mutex_t lock;
} vm_registry_t;

```

Etape 3.3 — Logique de recharge des credits

```

/* Appelee toutes les 30ms par timerfd */
void credit_recharge(vm_registry_t *reg) {
    pthread_mutex_lock(&reg->lock);
    for (int i = 0; i < reg->vm_count; i++) {
        vm_entry_t *vm = reg->vms[i];
        /* Credits proportionnels au weight */
        int64_t earned = (vm->weight * 100) / reg->total_weight;
        vm->credits += earned;
        vm->gpu_usage_ms = 0; /* reset slot */
        /* Mise a jour etat */
        vm->state = (vm->credits > 0) ? UNDER : OVER;
    }
}

```

```
pthread_mutex_unlock(&reg->lock);
}
```

Etape 3.4 — Compilation

```
gcc -Wall -Wextra -O2 -pthread \
-o gpu-scheduler \
src/main.c src/scheduler.c src/vm_registry.c \
src/job_queue.c src/config.c src/socket_server.c \
src/mps_bridge.c src/cli_server.c src/metrics.c \
-lyaml -lpthread
```

Phase 4 — Packaging .deb

Etape 4.1 — Structure du package

```
gpu-remoting_1.0.0/
DEBIAN/
control          # metadonnees package
postinst         # script post-installation
prerm            # script pre-suppression
usr/bin/
gpu-scheduler
gpu-cli
usr/lib/gpu-remoting/
cricket-rpc-server
cricket-client.so
etc/gpu-remoting/
config.yaml
lib/systemd/system/
gpu-remoting.service
```

Etape 4.2 — Construire le .deb

```
dpkg-deb --build gpu-remoting_1.0.0/
# Produit : gpu-remoting_1.0.0.deb
```

Etape 4.3 — Installation sur host Proxmox

```
dpkg -i gpu-remoting_1.0.0.deb
systemctl enable gpu-remoting
systemctl start gpu-remoting
gpu-cli status
```

6. Etat d'avancement actuel

Driver NVIDIA	En cours — installation via .run NVIDIA sur kernel 6.17.x-pve
Cricket compilation	Non commence — apres validation driver
Test Cricket local	Non commence

Test Cricket VM	Non commence
Scheduler C	Conception complete — implementation a venir
Package .deb	Structure definie — a venir en phase 4
Monitoring	Prevu — optionnel phase finale

Probleme en cours : le kernel Proxmox 6.17.2-1-pve est trop recent pour les headers pve dans les repos stables. Solution retenue : installation du driver NVIDIA via le fichier .run avec le flag --dkms pour recompilation automatique a chaque changement de kernel.

7. Limitations connues

Limitation	Impact / Mitigation
Latence reseau	Overhead par rapport au GPU local — acceptable pour batch ML, insuffisant pour temps reel strict
Support CUDA partiel	Cricket ne supporte pas les fonctions graphiques CUDA (non pertinent pour notre usage HPC/ML)
GTX 1650 Ti Mobile	GPU laptop — pas de NVLink, VRAM limitee a 4 Go — suffisant pour tests et prototype
Un seul GPU	Pas de multi-GPU en phase 1 — prevu en roadmap phase 3
Securite reseau	Isolation a configurer manuellement — authentication Cricket a activer en production

8. Roadmap

- **Phase 1 (actuelle) : Driver NVIDIA + Cricket operationnel**
 - Resoudre installation driver sur kernel 6.17.x
 - Valider nvidia-smi sur host
 - Compiler Cricket et tester en local
 - Tester depuis une VM Ubuntu
- **Phase 2 : Scheduler Credit en C**
 - Implementer vm_registry, job_queue, credit_engine
 - Integrer CUDA MPS pour le cap
 - Tester avec 2 puis 3 VMs simultanees
- **Phase 3 : Optimisations et monitoring**
 - Prometheus exporter en C
 - Dashboard Grafana
 - Optimisation reseau (TCP tuning)

- **Phase 4 : Package .deb et documentation finale**
 - Script postinst complet
 - Tests d'integration automatises
 - Documentation utilisateur

GPU-as-a-Service — Projet en cours de developpement